

Fee Management System on Microsoft Azure

Deployment Guide

1. Project Overview

The Fee Management System is a cloud-based application developed using Microsoft Azure services. The solution enables students to check their fee payment status, administrators to securely update fee records, and automatically sends reminder emails to students with overdue payments.

The project uses Azure Functions for serverless APIs, Azure SQL Database for persistent storage, Azure API Management for secure API exposure, Azure Logic Apps for workflow automation, Microsoft Entra ID (Azure AD) for authentication and Role-Based Access Control (RBAC), and Application Insights for monitoring and diagnostics.

2. Technologies Used

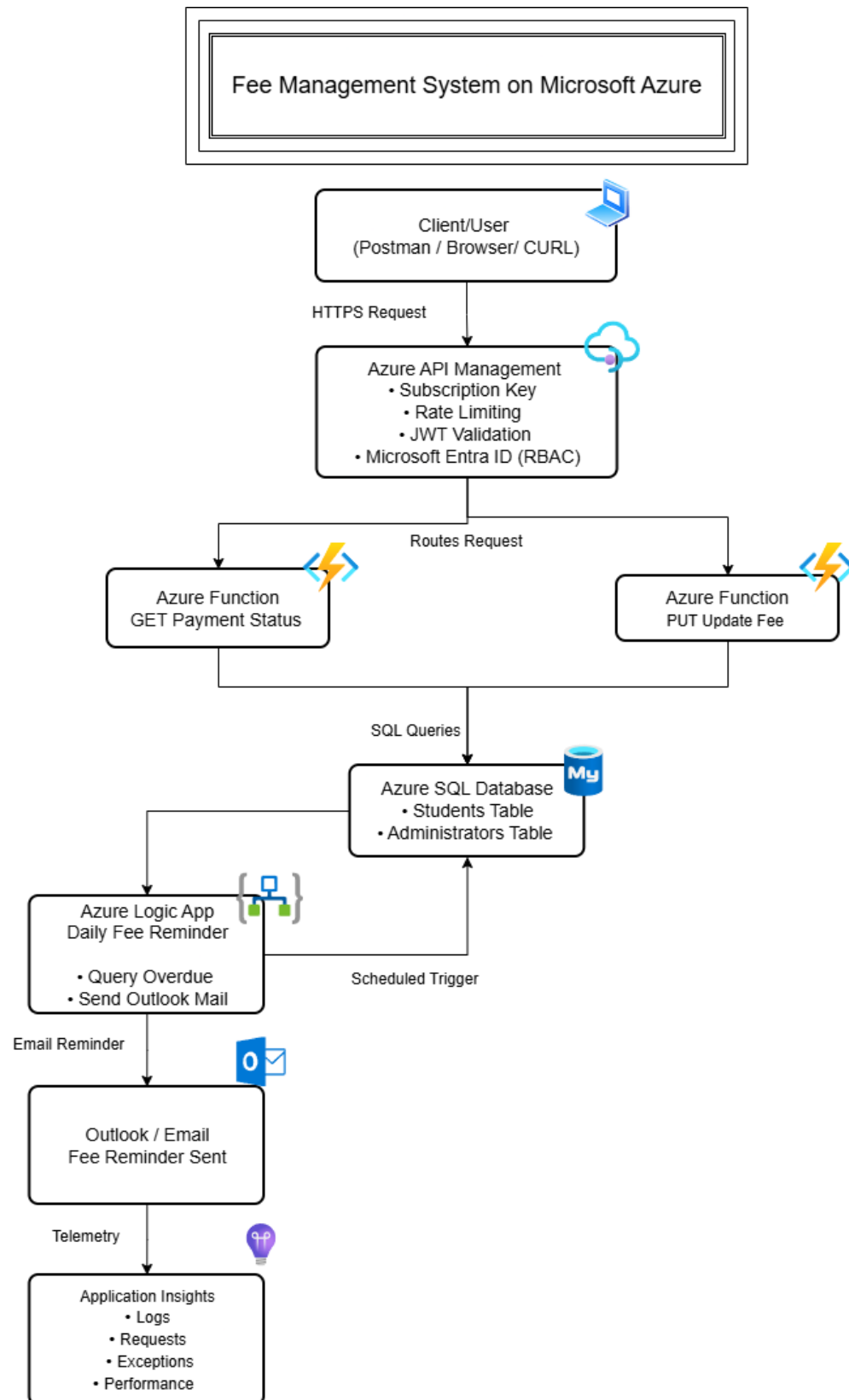
Programming Languages

- Python 3.12
- SQL

Microsoft Azure Services

- Azure SQL Database
- Azure SQL Server
- Azure Functions
- Azure API Management
- Azure Logic Apps
- Microsoft Entra ID (Azure AD)
- Azure Application Insights
- Azure Storage Account

Project Architecture Diagram:



Development Environment

- Ubuntu (WSL)
 - Visual Studio Code
 - Azure Portal
 - Git
 - Postman
 - curl
-

3. Required Software Installation

Install the following software before starting the project

Software	Purpose
Python 3.12	Backend Development
Visual Studio Code	IDE
Git	Version Control
Azure CLI	Azure Management
Azure Functions Core Tools	Local Function Execution
ODBC Driver 18 for SQL Server	SQL Connectivity
Microsoft SQL Server VS Code Extension	Database Management
Azure VS Code Extension	Azure Deployment
Python Extension	Python Development
Pylance	Python Language Support

4. Installation and Environment Setup

Step 1: Install Python

Verify Python installation.

```
python3 --version
```

or

```
python --version
```

Step 2: Install Git

Ubuntu

```
sudo apt update
```

```
sudo apt install git
```

Verify

```
git --version
```

Step 3: Install Visual Studio Code

Download and install Visual Studio Code from:

<https://code.visualstudio.com/>

Step 4: Install Required VS Code Extensions

Install the following extensions:

- Python
 - Pylance
 - Azure Functions
 - Azure Resources
 - SQL Server (mssql)
-

Step 5: Install Azure CLI

Ubuntu

```
curl -sL https://aka.ms/InstallAzureCLIDeb | sudo bash
```

Verify

```
az version
```

Login

```
az login
```

Step 6: Install Azure Functions Core Tools

Ubuntu

```
sudo apt update
```

```
sudo apt install azure-functions-core-tools-4
```

Verify

```
func --version
```

Step 7: Install ODBC Driver 18

Ubuntu

```
sudo apt update
```

```
curl https://packages.microsoft.com/keys/microsoft.asc | sudo apt-key add -
```

```
curl https://packages.microsoft.com/config/ubuntu/22.04/prod.list \
```

```
| sudo tee /etc/apt/sources.list.d/mssql-release.list
```

```
sudo apt update
```

```
sudo ACCEPT_EULA=Y apt install msodbcsql18
```

Verify

```
odbcinst -q -d
```

Step 8: Clone the Repository

```
git clone <REPOSITORY_URL>
```

```
cd Fee-Management-Service
```

Step 9: Create Virtual Environment

```
python3 -m venv .venv
```

Activate

```
source .venv/bin/activate
```

Step 10: Install Python Dependencies

```
pip install -r requirements.txt
```

Step 11: Create local.settings.json

Create

local.settings.json

Example

```
{  
  "IsEncrypted": false,  
  "Values": {  
    "AzureWebJobsStorage": "<AZURE_STORAGE_CONNECTION_STRING>",  
    "FUNCTIONS_WORKER_RUNTIME": "python",  
    "SQL_CONNECTION_STRING": "<AZURE_SQL_CONNECTION_STRING>"  
  }  
}
```

```
}
```

Step 12: Run the Azure Function Locally

```
func start
```

Step 13: Deploy to Azure

```
az login
```

```
func azure functionapp publish <FUNCTION_APP_NAME>
```

5. Create Azure Resources

Create the following Azure resources in the given order.

1. Resource Group
 2. Azure SQL Server
 3. Azure SQL Database
 4. Azure Storage Account
 5. Azure Function App
 6. Azure Application Insights
 7. Azure API Management
 8. Azure Logic App
 9. Microsoft Entra App Registrations
-

6. Database Setup

Step 1

Connect Visual Studio Code to the Azure SQL Database using the SQL Server extension.

Connection Details

- Server Name
- Database Name
- Username
- Password

Step 2

Execute the provided SQL script.

Database/database_setup.sql

The script performs the following operations:

- Creates the Students table
- Creates the Administrators table
- Inserts sample administrator records
- Inserts sample student records
- Contains SQL queries used throughout the application

7. Azure Function Setup

Clone the repository.

```
git clone <repository-url>
```

Navigate into the project.

```
cd Fee-Management-Service
```

Activate the environment.

Linux

```
source .venv/bin/activate
```

Windows

```
.venv\Scripts\activate
```


Install dependencies.

```
pip install -r requirements.txt
```

Configure local.settings.json

Create a file named

local.settings.json

Example configuration

```
{  
  "IsEncrypted": false,  
  "Values": {  
    "AzureWebJobsStorage": "<Storage Connection String>",  
    "FUNCTIONS_WORKER_RUNTIME": "python",  
    "SQL_CONNECTION_STRING": "<Azure SQL Connection String>"  
  }  
}
```

Run Function Locally

```
func start
```

8. API Management Configuration

Import the Azure Function into Azure API Management.

Create two operations.

GET and PUT

1. GET

/paymentstatus/{studentid}

Purpose:

Returns

- Student Details
 - Fee Details
 - Payment Status
-

2. PUT

/updatefee/{studentid}

Purpose:

Allows administrators to update the paid fee amount.

Configure Subscription Key Authentication

Enable

- Subscription Required

Generate

- Primary Subscription Key
- Secondary Subscription Key

Configure Rate Limiting

Add Rate Limit policy.

Example: 10 requests per minute

Configure JWT Validation

Add Validate JWT policy.

Configure

- OpenID Configuration URL
- Audience
- Required Claim
- Role = Admin

9. Microsoft Entra ID Configuration

Register API Application

Create an App Registration named
fee-management-admin-api

Expose API

Create access_api scope.

Create App Role

Role Admin

Allowed Member Types Applications

Register Client Application

Create a Client App Registration named fee-management-api-client
and

Generate

- Client Secret

Grant API Permissions.

Grant Admin Consent.

Configure RBAC

Assign Admin Application Role to fee-management-api-client

Update API Management

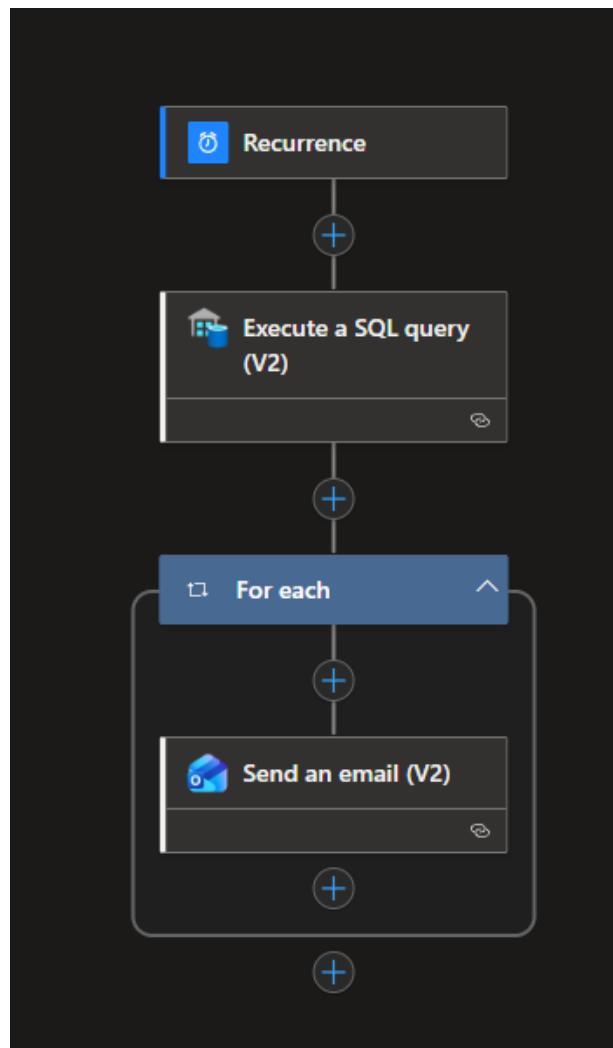
Configure Validate JWT policy using

- OpenID Configuration
- Audience
- Required Claims

10. Logic App Configuration

Create a Logic App.

Workflow



SQL Query

Retrieve students whose

- PaidAmount < TotalFee

and

- DueDate < Current Date

Outlook Action

Send reminder email containing

- Student Name
- Outstanding Fee
- Due Date

Retry Policy

Configure retry policy on the SQL Query action.

Recommended

- Retry Count = 3
 - Minimum Interval = 5 Seconds
 - Maximum Interval = 45 Seconds
-

11. Application Insights

Enable Application Insights during Function App creation.

Use it to monitor

- Incoming Requests
 - Response Time
 - Exceptions
 - Failed Requests
 - Dependency Calls
-

12. Testing the Application

Test Payment Status API

GET /paymentstatus/{studentid}

Expected Result

Returns payment information and payment status.

Test Update Fee API

PUT /updatefee/{studentid}

Expected Result

Updates student payment record.

Test Unauthorized Access

Invoke Update API

without JWT.

Expected Result

401 Unauthorized

Test Authorized Access

Invoke Update API

using

- Bearer Token
- Subscription Key

Expected Result

200 OK

Test Rate Limiting

Invoke GET API repeatedly.

Expected Result

429 Too Many Requests

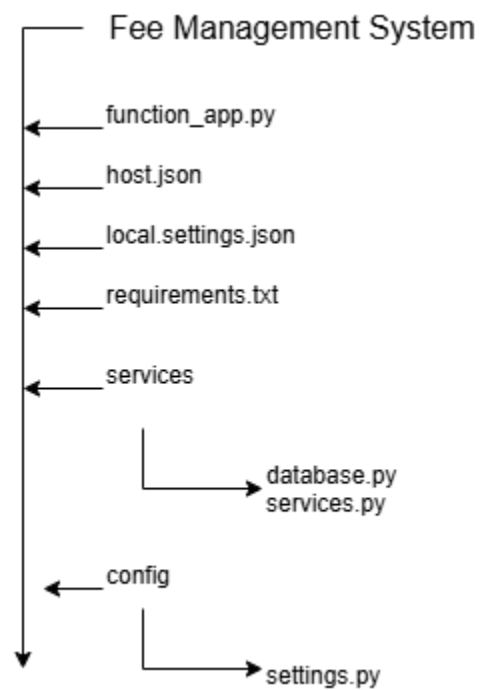
after exceeding the configured request limit.

Test Logic App

Trigger Logic App manually or wait for the recurrence schedule.

Verify reminder emails are sent to students with overdue payments.

13. Project Structure of main execution files:



14. Security Considerations

The following files should **not** be committed to source control.

- local.settings.json
- Client Secrets
- Subscription Keys
- SQL Connection Strings
- Bearer Tokens

Instead, use placeholder values in shared configuration files.

15. Conclusion

The Fee Management System demonstrates a secure and scalable serverless architecture using Microsoft Azure services. The solution integrates Azure Functions, Azure SQL Database, Azure API Management, Azure Logic Apps, Microsoft Entra ID, and Application Insights to provide secure fee management, automated reminders, and comprehensive monitoring.

16. Appendix

curl commands

Test Rate Limiting and API Key Authentication

Replace <SUBSCRIPTION_KEY> with your own API Management subscription key.

```
for i in {1..12}
do
    curl -i \
        "https://<APIM_NAME>.azure-
api.net/<API_NAME>/paymentstatus/<STUDENT_ID>?subscription-key=<SUBSCRIPTION_KEY>"
    echo
done
```

Test Update Fee API (Local)

```
curl -X PUT \  
-H "Content-Type: application/json" \  
-d '{"PaidAmount":95000}' \  
http://localhost:7071/api/updatefee/<STUDENT_ID>
```

Test paymentstatus route

```
curl http://localhost:7071/api/paymentstatus/<STUDENT_ID>
```

Test PUT Operation through Azure API Management

In the API Management Test Console, provide the following request body:

```
{  
  "PaidAmount": <AMOUNT>  
}
```

Get Access token

Replace the placeholder values with your own Microsoft Entra ID configuration.

```
curl -X POST \  
"https://login.microsoftonline.com/<TENANT_ID>/oauth2/v2.0/token" \  
-H "Content-Type: application/x-www-form-urlencoded" \  
-d "client_id=<CLIENT_APPLICATION_ID>" \  
-d "client_secret=<CLIENT_SECRET>" \  
-d "scope=api://<API_APPLICATION_ID>/default" \  
-d "grant_type=client_credentials"
```

Test Secure Admin API

Replace the placeholder values before executing the request.

```
curl -X PUT \  
"https://<APIM_NAME>.azure-api.net/<API_NAME>/updatefee/<STUDENT_ID>" \  
-H "Authorization: Bearer <ACCESS_TOKEN>" \  
-H "Ocp-Apim-Subscription-Key: <SUBSCRIPTION_KEY>" \  
-H "Content-Type: application/json" \  
-d '{"PaidAmount":110000}'
```

Expected Outcome

- ✓ Student fee information retrieved successfully
 - ✓ Fee records updated only by authenticated administrators
 - ✓ Overdue fee reminders sent automatically
 - ✓ Application requests monitored through Application Insights
 - ✓ Rate limiting enforced through Azure API Management
-